SYSTEM DOCUMENTATION

Prepared by
GROUP 3

https://scout-delta-ruby.vercel.app/

scout.communications@yahoo.com

Project members, roles and overview

Project manager and full-stack engineer	Mathabo Mabunu
Backend developers	Bonginkosi Lutshetu Boitumelo Madela
Frontend developers	Keanu Madasi Oghenetega Zintle Mabandlela
Software testers	Oghenetega Zintle Mabandlela, Mihle Madikane
Submission Type	Term 4 Group
Stack	React (Vite) and Tailwind, Django REST Framework, PostgreSQL on Supabase, Brevo SMTP
Official website link	https://scout-delta-ruby.vercel.app/

Table of Contents

1. Introduction and system requirements..... 5
2. Use case diagram..... 13
3. Application architecture..... 14
4. Activity diagrams..... 17
5. Class diagram..... 19
6. Systems sequence diagrams..... 20
7. Database design..... 23
8. Frontend structure..... 25
9.Setup, maintenance and troubleshooting..... 26
10. Testing summary..... 28

1. Introduction and system requirements

Scout is a web platform that helps tertiary students and graduates find opportunities they qualify for, apply with one reusable profile, and track outcomes in one place. Providers post opportunities, students browse them, and a platform administrator verifies every listing before students see it. The system exists because students told us through primary data collection that they frustratingly re-enter the same documents repeatedly, scroll past listings they cannot apply for, and check several portals for outcomes.

1.1 Functional requirements

Account and access

ID	Requirement
FR1	A student registers with full name, email and a password of at least 10 characters. A provider registers with company name, email, password, industry, province and an optional phone number. The email must be unique across students, providers and auth users. The account is created inactive.
FR2	The system emails a six-digit code on registration. The correct code activates the account and signs the user in immediately. The code is valid for 5 minutes, allows 5 entry attempts and 3 resends, with a 60-second cooldown. Only a salted hash of the code is stored.

FR3	A verified user signs in with email and password alone. No code is required on login. An unverified account that supplies the correct password is told to complete verification rather than being given a generic credentials error.
FR4	A user signs out. The server deletes the authentication token so it cannot be reused.
FR5	Three roles exist: student, provider and administrator. Each has its own routes and permitted actions, enforced on the server. Requesting another user's record returns not-found, so users cannot discover each other's data.

Profile and documents

ID	Requirement
FR6	A student records institution, field of study, academic average, opportunity preference and province. A Tertiary Student adds year of study. A Graduate adds graduate type and qualification. The database rejects inconsistent combinations.
FR7	The profile holds professional information only. ID numbers, home address, banking details, health information, race and salary expectations are never collected.
FR8	A student uploads a CV and matric certificate (both required before applying), plus optional transcripts, proof of registration and supporting documents. Accepted types are PDF, JPG, JPEG and PNG up to 5 MB. Replacing a document retires the previous version instead of deleting it.
FR9	Files are stored privately in Supabase Storage and reached through signed links that expire after one hour. The storage key never reaches the browser.

Opportunities and matching

ID	Requirement

FR10	The server calculates eligibility from the student's stored profile, never from values sent by the browser. The full rule is in section 3.4.
FR11	A student switches between an Eligible view and an All view. Both exclude unverified, inactive and expired listings, and both are ordered by closing date. In the All view each listing is flagged with whether the student qualifies.
FR12	Each listing shows whether the student has already bookmarked or applied to it.

Applying

ID	Requirement
FR13	The system pre-fills an application from the stored profile and attaches the student's active documents. A review screen is mandatory. Nothing is submitted until the student confirms on that screen.
FR14	On confirmation the system creates the application with status *Submitted* and a unique reference number..
FR15	An application is refused when the opportunity has closed, the student is not eligible, the profile is incomplete, a required document is missing, or the student has already applied.
FR16	When a required document is missing, the system lists what is missing, blocks submission, and notifies the student in the application and by email with a link back to the same review screen. The alert is sent once, so reopening the screen does not send repeat emails.
FR17	Each submission produces a receipt with the student's name, opportunity, provider, submission date, reference number and status. It appears on the dashboard and is emailed.
FR18	A student downloads a PDF summary of all applications, with totals by status and every reference number.

Tracking and notifications

ID	Requirement
FR19	A dashboard lists every application with its status: Submitted, Under Review, or Outcome Received. It refreshes automatically every 15 seconds.
FR20	A student saves opportunities to a watchlist and removes them again. The watchlist shows closing dates. Saving the same opportunity twice has no additional effect.
FR21	The system sends a reminder three days before a bookmarked opportunity closes, once per bookmark. Students who have already applied are skipped.
FR22	The student is notified when an application is submitted, a document is missing, a status changes, an outcome is recorded, and a bookmarked deadline approaches. Notifications appear in the application and, where appropriate, by email. The student can mark one or all as read.

Provider

ID	Requirement
FR23	A provider creates, edits and removes its own listings, and may set optional eligibility criteria. Every provider query is restricted to that provider's own records.
FR24	A new listing is created unverified and is not visible to students until an administrator verifies it.
FR25	A provider sees applicants for its own listings, each with their professional profile and which required documents are attached. File links are not exposed to providers.
FR26	A provider moves an application to Under Review and then records an outcome of Successful or Unsuccessful. Status only moves forward, and an outcome once recorded cannot be changed.

Administration and privacy

ID	Requirement

FR27	Every new listing enters a verification queue. An administrator reviews the queue, opens a listing's full detail and verifies it.
FR28	Verification makes the listing visible to matching students and records the verifying administrator against the listing as an audit trail. A listing already verified cannot be verified again.
FR29	Providers see only a student's professional profile. A student can view everything stored about them through the profile and documents screens.
FR30	POPIA also grants rights to restrict and delete personal data.

1.2 Non-functional requirements

ID	Requirement	How it is met
NFR1	Credential security	Passwords hashed by Django. Verification codes and resend tokens stored as salted hashes. Neither appears in logs or API responses. Secrets live in environment files excluded from version control.
NFR2	Transport and storage security	Database connection requires SSL. HTTPS redirection and secure cookies enabled whenever debug is off. Documents are private, reached only through signed links expiring after one hour.
NFR3	Authentication hardening	Verification and resend limited to 5 requests per minute per client. Anonymous requests 20 per minute, authenticated 120 per minute. Code entry limited to 5 attempts, 3 resends, 60-second cooldown.
NFR4	Input and query safety	All raw SQL uses parameter placeholders. Fixed value lists in query text comes from server-side constants, never user input. Uploads checked for type and size. Internal errors logged, never echoed to the client.

NFR5	Data integrity	Unique constraints prevent duplicate applications and bookmarks at database level, not interface level. Profile field combinations enforced by database constraints. Status transitions validated server-side and cannot move backwards.
NFR6	Auditability	Each verified listing records which administrator verified it. Every provider status change is written to the application log with the provider and a timestamp
NFR7	Notification timeliness	The dashboard polls every 15 seconds, so a provider's change reaches the student within roughly that interval. Emails are sent as soon as the triggering action completes. A mail failure is logged and never blocks the underlying action.
NFR8	Usability	Works across desktop and tablet widths. Validation runs in the browser first, with server errors shown in one plain-language message. Loading states shown while fetching. Animations respect the reduced motion setting.
NRF9	Performance	Listing results come from a single database query, paginated at 20 records per page.
NFR10	Reliability	Managed hosting for database, file storage and email.

1.3 User stories

	Story	Acceptance

US1	As a student, I want to register and confirm my email with a code, so that my account is protected and I can be contacted about applications.	Registration creates an inactive account and emails a six-digit code. The correct code activates the account and signs me in.
US2	As a student, I want to sign in with just my email and password after the first time, so that getting back in is quick.	Login requires no code. An unverified account is told to verify first.
US3	As a student, I want to enter my academic details and upload my CV and matric certificate once so that I do not retype them for every application.	Profile and documents are saved once and reused. Replacing a document retires the old version.
US4	As a student, I want to see only the opportunities I actually qualify for, so that I do not waste time on listings that will reject me.	The Eligible view applies every criterion the provider sets. The All view shows everything live, flagged with whether I qualify.
US5	As a student, I want the application form filled in from my profile with a review screen before it sends, so that I stay in control of what is submitted.	Nothing is submitted until I confirm on the review screen. I get a reference number afterwards.
US6	As a student, I want to be told when a required document is missing, with a link back to where I was, so that I can finish without starting over.	Review lists what is missing, blocks submission, and sends one alert with a link back to the same screen.
US7	As a student, I want all my applications and their statuses on one dashboard, so that I do not check several company portals.	The dashboard lists every application and refreshes every 15 seconds when a provider changes a status.
US8	As a student, I want to save opportunities I am not ready for and be reminded before they close, so that I do not miss a deadline.	Bookmarks appear on the watchlist with closing dates. A reminder is sent three days before closing, once per bookmark.
US9	As a student, I want to download a summary of everything I applied for, so that I have a record for bursary or university paperwork.	A PDF lists every application with status totals and reference numbers.

US10	As a provider, I want to post a listing and see who applied, so that I can fill the position without building my own portal.	I create listings under my own account, see only my own, and view each applicant's professional profile and attached documents.
US11	As a provider, I want to record an outcome so the student is told automatically, so that I do not send individual emails.	Setting an outcome to Successful or Unsuccessful notifies the student in the application and by email.
US12	As an administrator, I want to check listings before students see them, so that unverified content never reaches students.	New listings enter a queue. Verifying one makes it visible and records me against it.

1.4 System requirements

Component	Requirement
End user	Any current browser (Chrome, Edge, Firefox, Safari), an internet connection, and an email address they can access, since an account cannot be activated without receiving the verification code
Developer machine	Node.js 18 or newer, Python 3.11 or newer, Git, a code editor like VSCode
Backend libraries	Django 6.1, djangorestframework 3.18, psycopg2-binary, python dotenv, django-cors-headers, requests, reportlab (PDF summary), pillow
Frontend libraries	React, Vite, Tailwind CSS, react-router-dom, lucide-react
External services	Supabase project (PostgreSQL and Storage), Brevo account for SMTP, Yahoo email account

2. Use case diagram

The diagram shows three human actors and one secondary actor. The student uses the account, profile, browsing, application and tracking features. The provider registers, manages its own listings, reviews applicants and records outcomes. The administrator verifies listings so that unchecked content never reaches students. The email service delivers verification codes, receipts, alerts and reminders.

Registration includes email verification by six-digit code. Resending a code extends verification. Applying includes the mandatory review step, and extends into document upload when a required document is missing. Posting a listing includes the administrator verification queue.

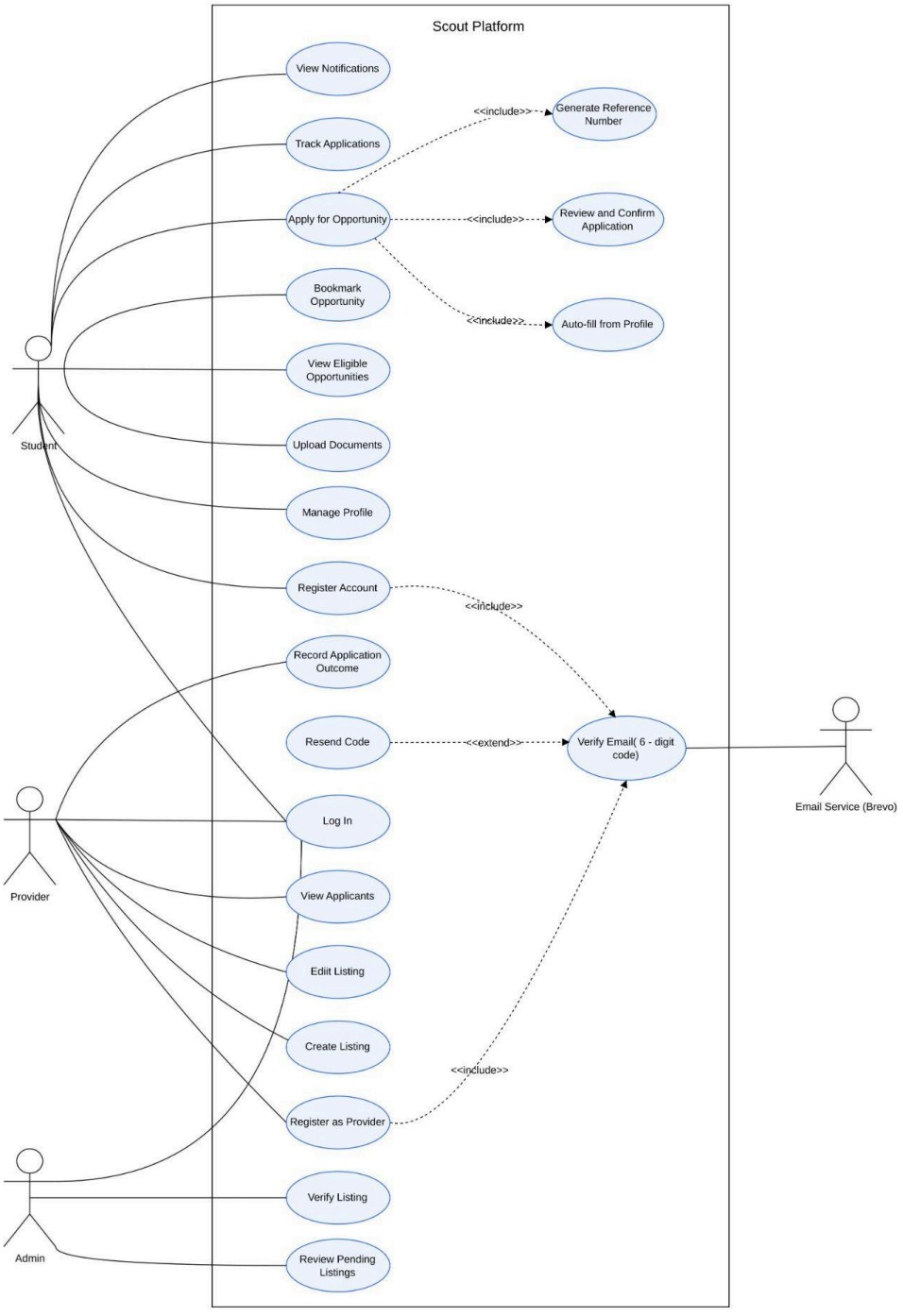


Figure 1: Use case diagram for Scout.

3. Application architecture

Scout is a three-tier web application. The React frontend runs in the browser and talks to the Django REST API over HTTPS using JSON. Django holds all business rules and is the only component that touches the database, file storage and email. Supabase provides managed PostgreSQL and file storage. Brevo delivers outgoing email.

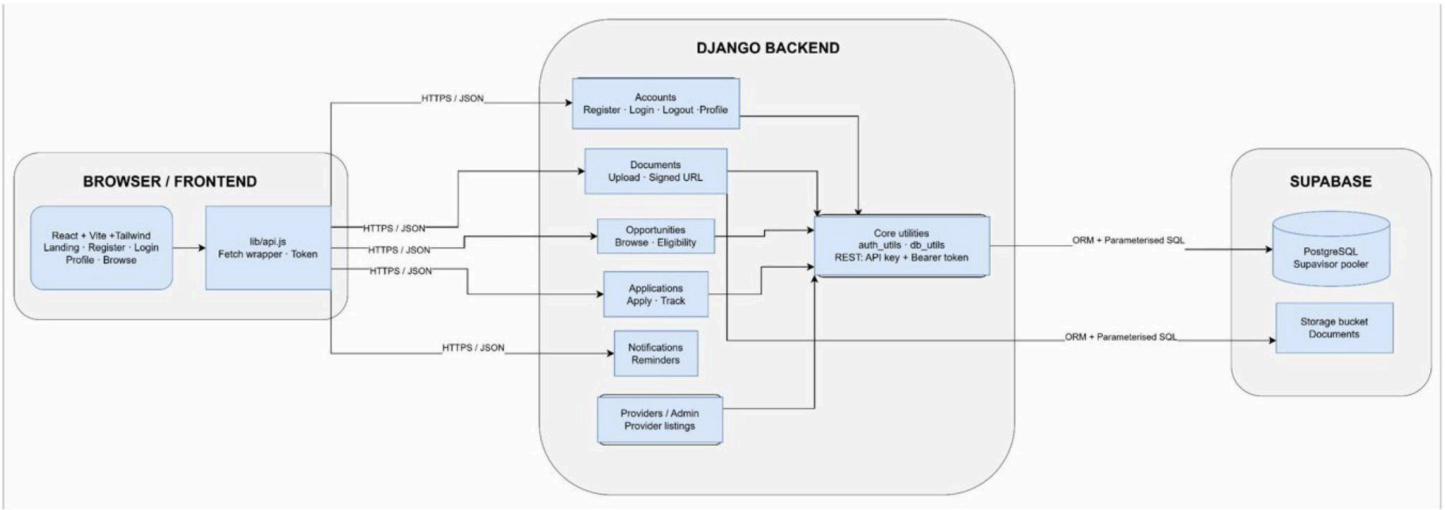


Figure 2: Application architecture.

3.1 Languages and technology choices

Layer	Technology	Reason
Frontend	JavaScript (React with Vite), Tailwind CSS, HTML	Component reuse, fast development, consistent styling. Chosen with AI assistance because the team had no prior JavaScript framework experience.

Backend	Python with Django and Django REST Framework	The team already knew Python. Django gives authentication, an ORM and a clear app structure.
Database	PostgreSQL on Supabase, SQL	Relational data with strict constraints. Managed hosting removes server upkeep.
File storage	Supabase Storage (REST)	Keeps documents out of the database. Signed links control access.
Email	Brevo SMTP, through Django's mail layer	A free tier that supports standard SMTP, so changing provider is a configuration change rather than a code change.
Version control	Git and GitHub	Pull request review for every push, which also taught the team CI/CD habits.

3.2 Key design decisions

- Django owns authentication, not Supabase Auth. Credentials stay in Django's *auth_user* table. Supabase Auth was considered and rejected: moving to it would have meant migrating every existing user, replacing token authentication across every protected endpoint, and running two authentication systems during the change. Supabase is used for database and file hosting only.
- Email verification happens once at signup, not on every login. An earlier build sent a code on every login. That was replaced because it adds friction to a routine action without much security gain once the email is confirmed. The accepted trade-off is that a stolen password alone is enough to sign in, which matches how most consumer platforms work.
- Hand-written schema with unmanaged models: Tables are created in Supabase from *database/schema.sql* . Django models set *managed = False* and map to lowercase columns with *db_column ,* so Django never alters the schema. The SQL file stays the single source of truth.
- Server-side eligibility: The matching query joins the student's profile from the authenticated user. The browser cannot claim a different average, field or qualification.
- ORM for CRUD, raw SQL for matching and reporting: The two reporting-style queries are clearer as SQL and use parameter placeholders throughout.
- Ownership checks instead of permission errors: Any object fetched for a user is filtered by both its own id and the owner id. A mismatch returns not-found, so a user cannot confirm that another user's record exists.
- Listings are gated at the listing level, not the account level. Providers self register without approval however, nothing they post reaches a student until an administrator verifies that specific listing.

3.3 Backend folder structure

Django app	Responsibility
scout_backend	Project shell: *settings.py , urls.py , wsgi.py*
core	Shared code: permissions.py (role checks for all three roles), *choices.py* (fixed value lists), *sql.py* and *db_utils.py* (raw SQL helpers), *auth_utils.py* , and *smoke_test* (end-to-end check)
accounts	Registration, email verification by code, login, logout, student profile. *otp_utils.py* holds code generation, hashing and expiry. *throttles.py* holds the verification rate limits
opportunities	Opportunity model, student browse views, provider listing management, and *matching.py* (the eligibility query). Management commands *seed_demo* and *verify_listings*
applications	Assisted apply, receipts, summary report, bookmarks, provider applicant review. *queries.p*y holds the reporting SQL, report_pdf.py builds the summary PDF
documents	Document model, upload and listing views, signed links, and *supabase_storage.py* (the storage adapter)
notifications	In-app and email notifications through *utils.py* , plus the *send_bookmark_reminders* command
providers_admin	Provider and PlatformAdmin models, the administrator verification queue and verify action

3.4 Eligibility rule

An opportunity is eligible for a student only when every criterion the provider specified passes. Three conditions apply before matching is considered. A criterion left blank places no restriction. In both instances, the listing must be verified, active, and not past its closing date.

Criterion	Rule
Field of study	Must equal the student's field of study

Opportunity type	Must equal the student's opportunity preference
Eligibility type	Any, or equal to the student's type (Tertiary Student or Graduate)
Minimum average	The student's average must be at or above it
Year level	Tertiary students only, at or above the required year. Graduates are excluded from listings that set one
Qualification	The student's qualification must be the required one or higher, ranked Higher Certificate, Diploma, Bachelor, Honours, Masters, Doctorate. A Masters graduate therefore qualifies for a Bachelor listing
Province	Must equal the listing's province

4. Activity diagrams

The following activity diagrams cover the main flows. Each decision diamond matches a validation or server check in the code.

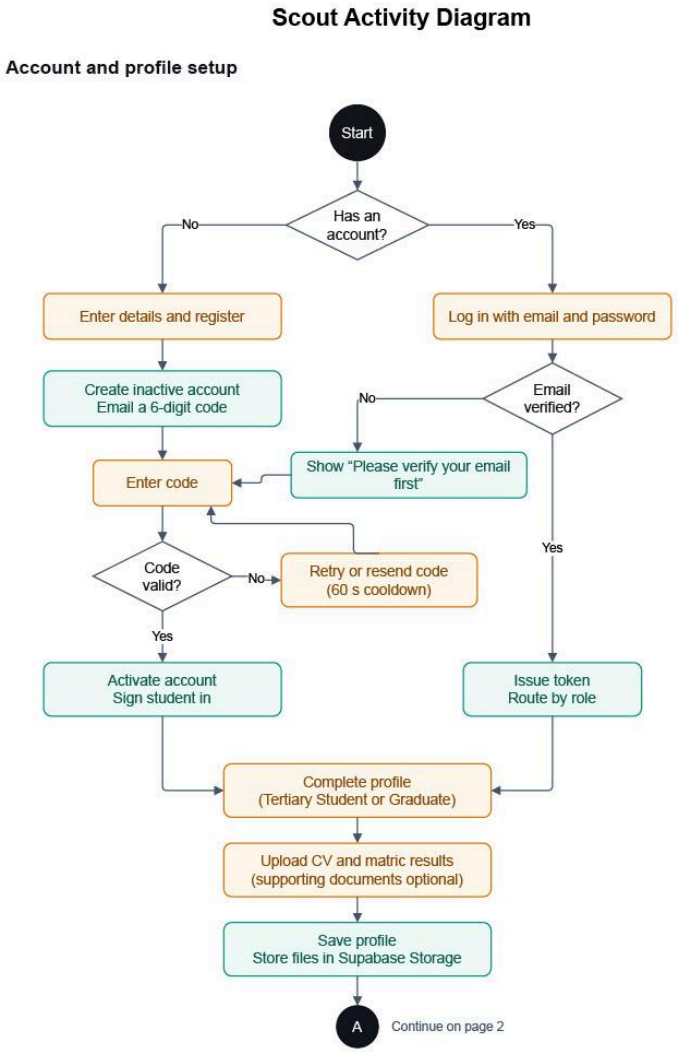


Figure 3: Account registration/login and profile set up.

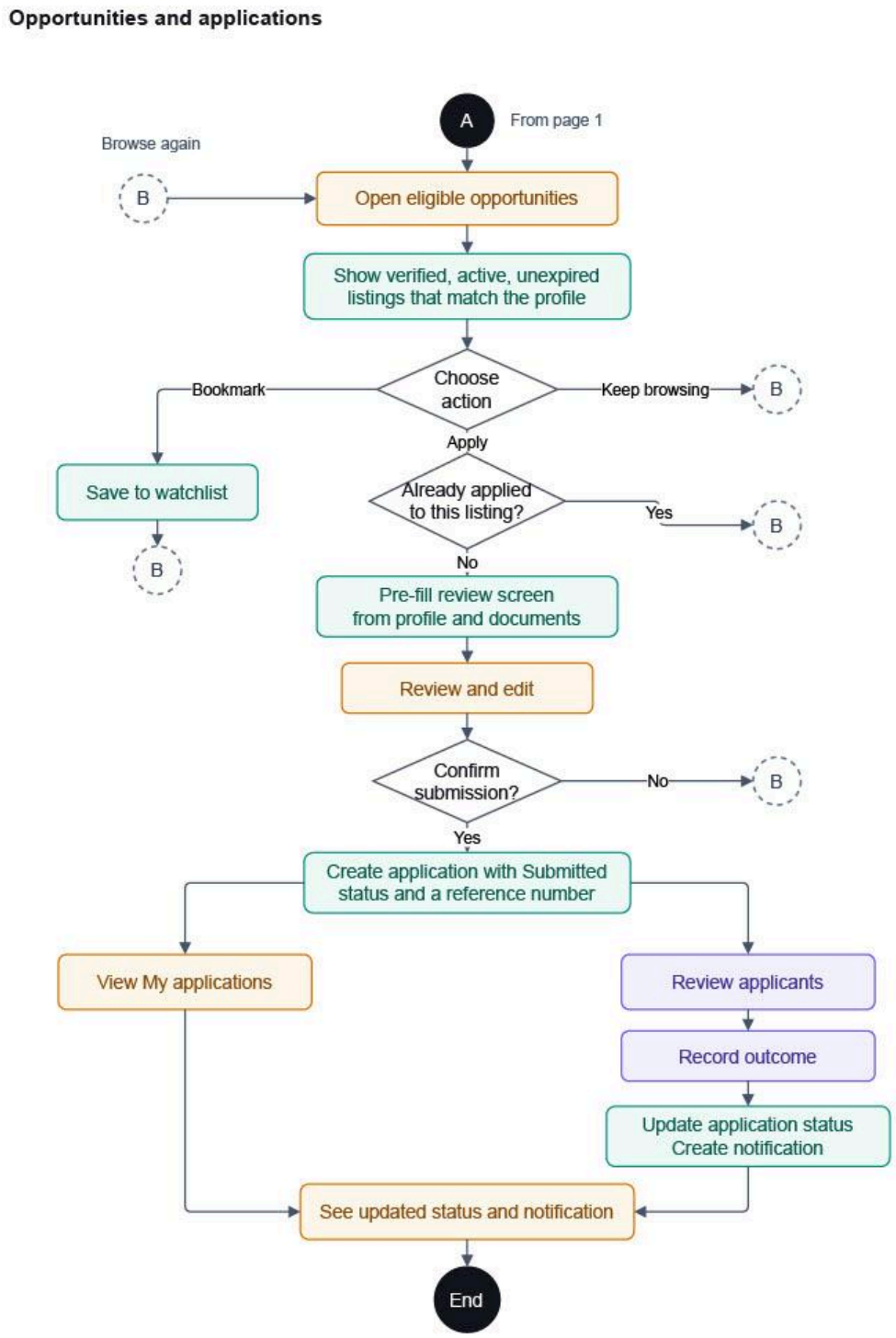


Figure 4: Browsing, eligibility and applying.

5. Class diagram

The classes mirror the database tables because each Django model maps to one table. Methods show behaviour that lives in views and helper modules rather than on the model itself.

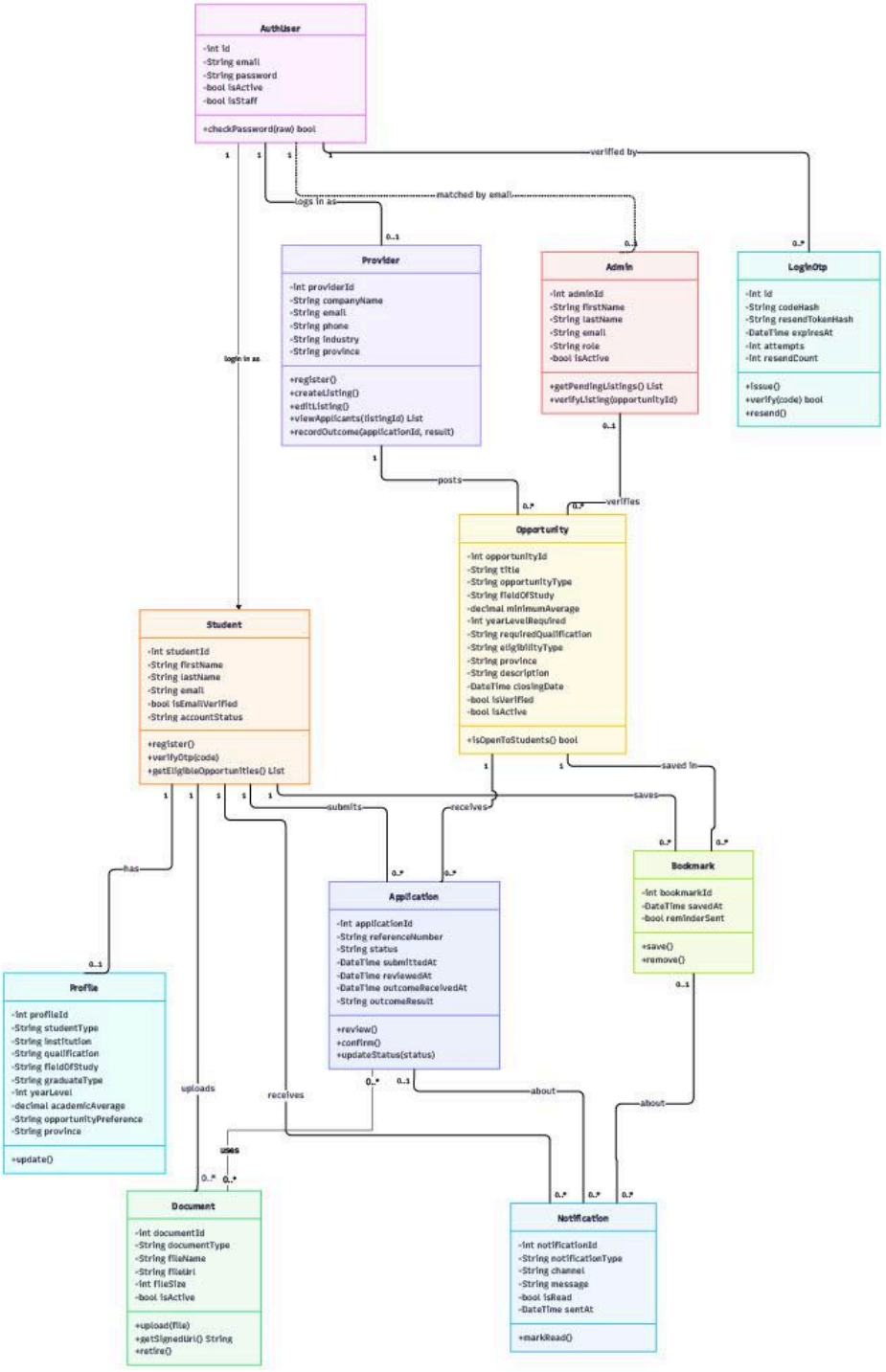
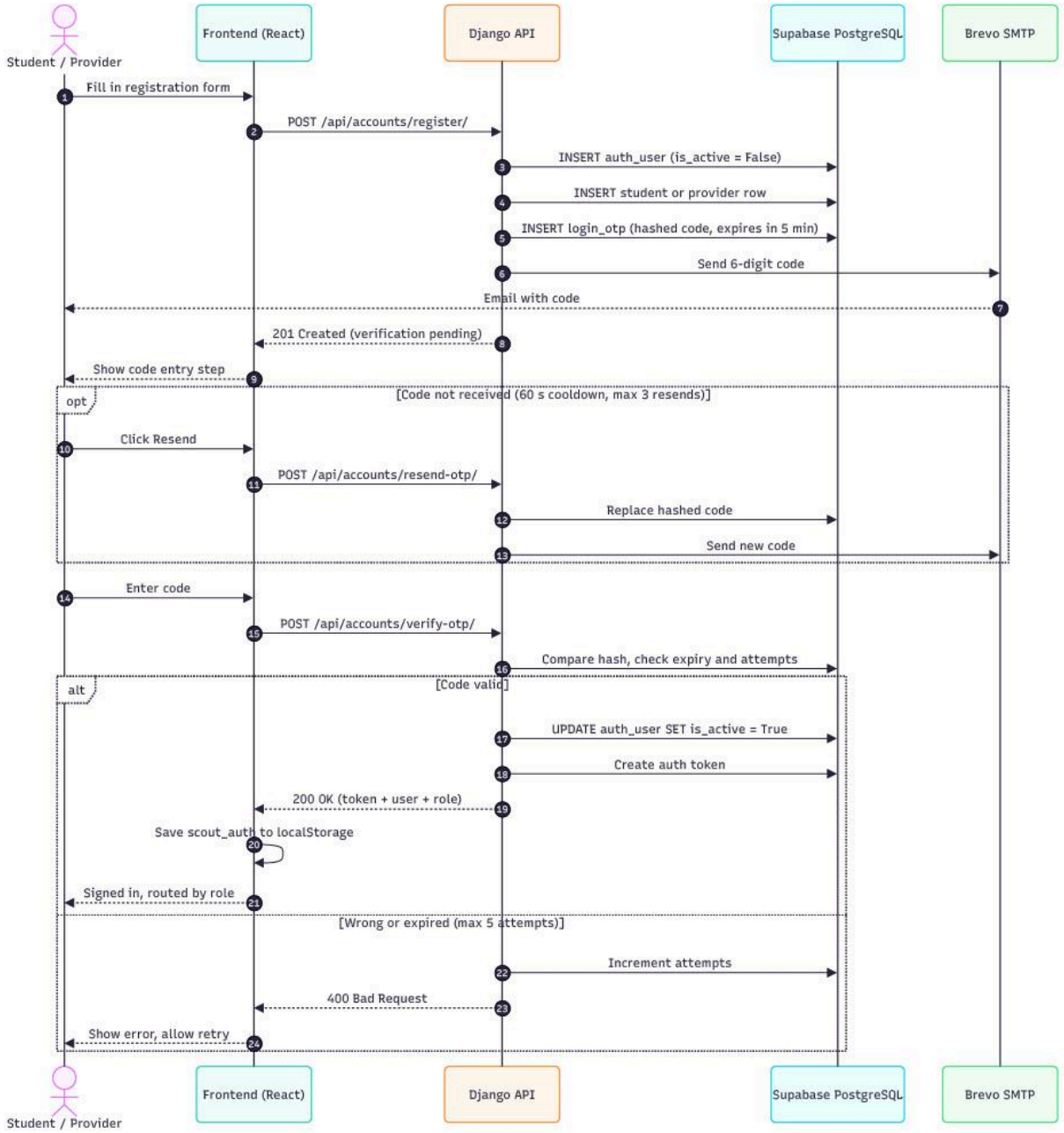
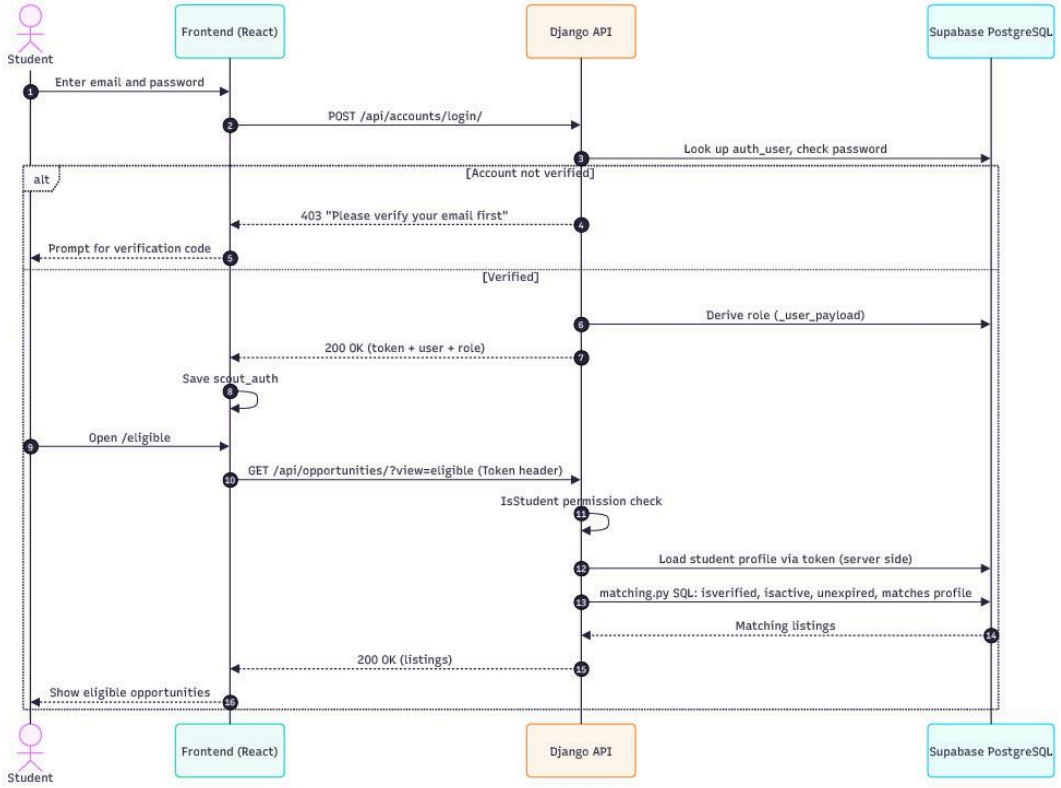
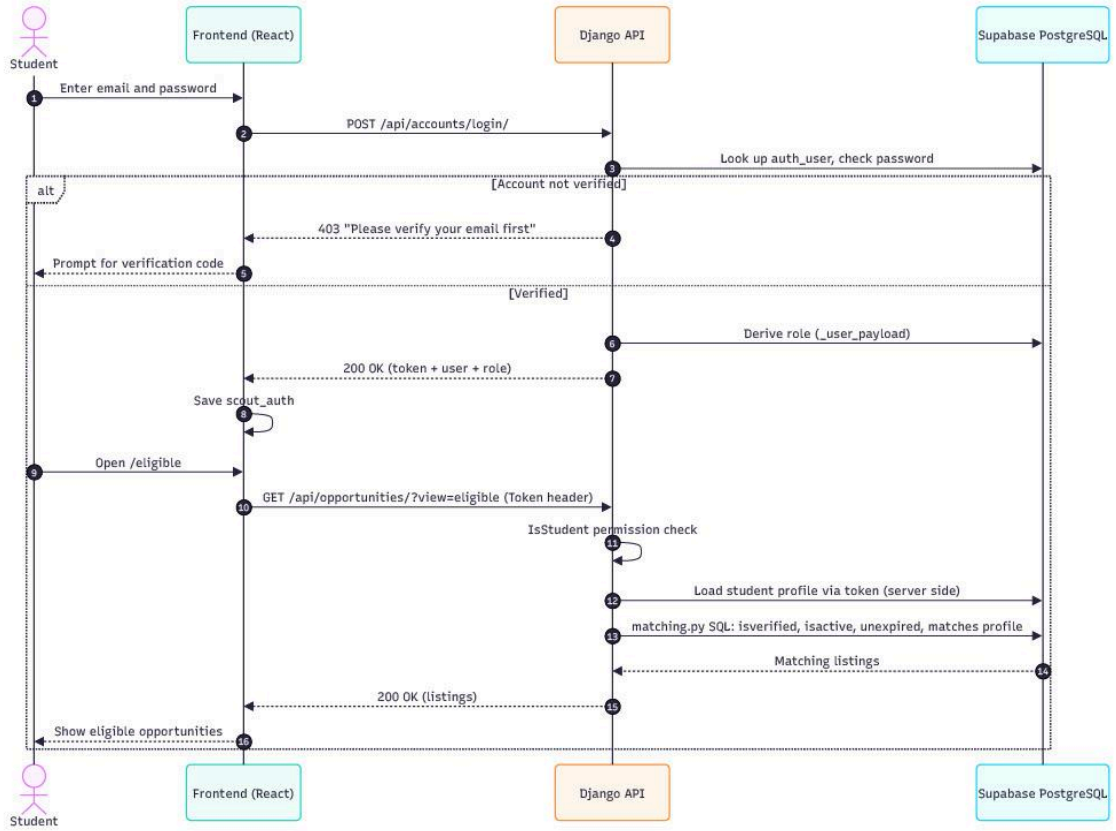


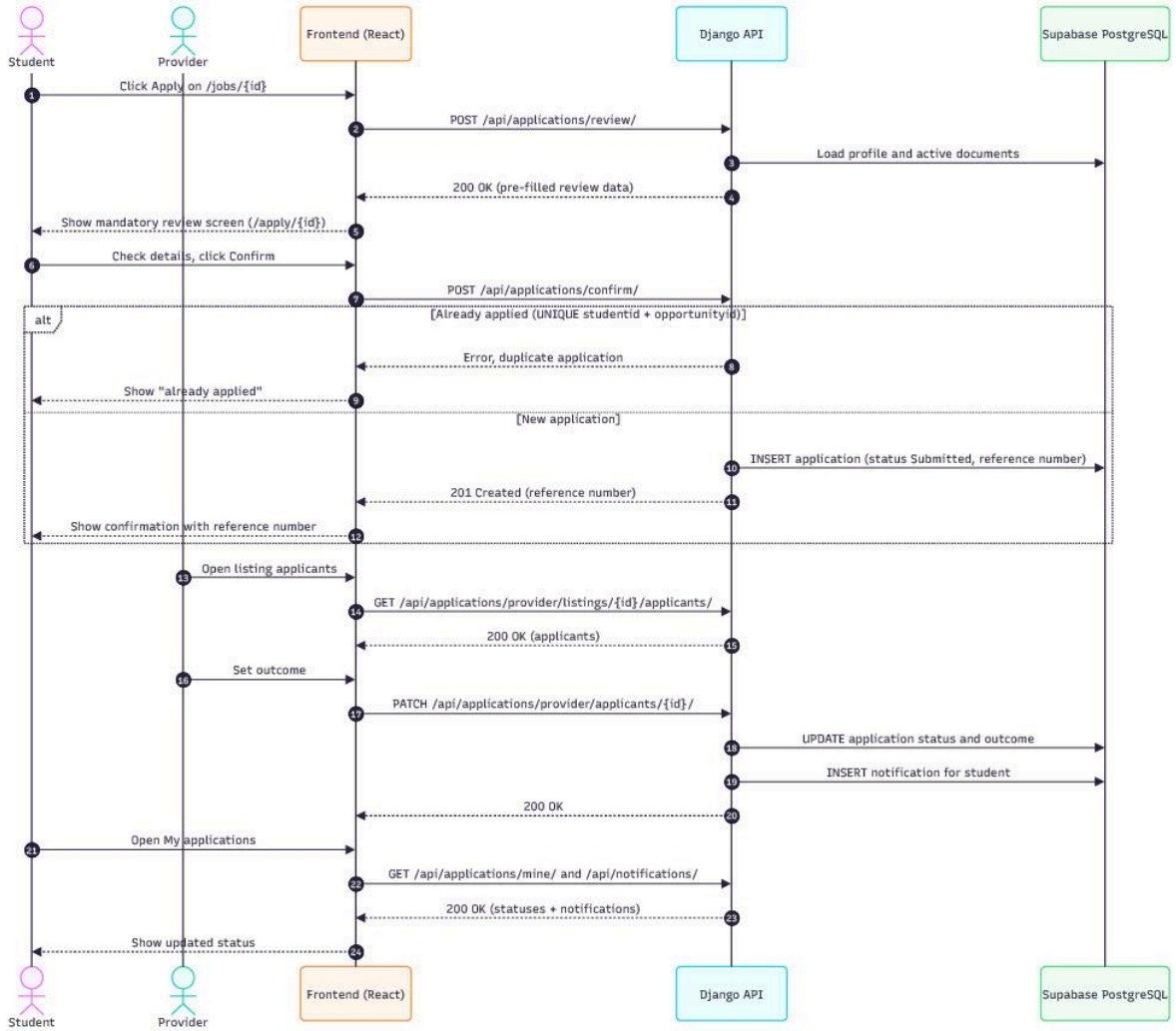
Figure 6: Class diagram.

6. Systems sequence diagrams




Student
Frontend (React)
Django API
Supabase PostgreSQL
1 Enter email and password
2 POST /api/accounts/login/
3 Look up auth_user, check password
alt
[Account not verified]
4 403 "Please verify your email first"
5 Prompt for verification code
[Verified]
6 Derive role (_user_payload)
7 200 OK (token + user + role)
8 Save scout_auth
9 Open /eligible
10 GET /api/opportunities/?view=eligible (Token header)
11 IsStudent permission check
12 Load student profile via token (server side)
13 matching.py SQL: isverified, isactive, unexpired, matches profile
14 Matching listings
15 200 OK (listings)
16 Show eligible opportunities
Student
Frontend (React)
Django API
Supabase PostgreSQL


Student
Frontend (React)
Django API
Supabase PostgreSQL
1 Enter email and password
2 POST /api/accounts/login/
3 Look up auth_user, check password
alt
[Account not verified]
4 403 "Please verify your email first"
5 Prompt for verification code
[Verified]
6 Derive role (_user_payload)
7 200 OK (token + user + role)
8 Save scout_auth
9 Open /eligible
10 GET /api/opportunities/?view=eligible (Token header)
11 IsStudent permission check
12 Load student profile via token (server side)
13 matching.py SQL: isverified, isactive, unexpired, matches profile
14 Matching listings
15 200 OK (listings)
16 Show eligible opportunities
Student
Frontend (React)
Django API
Supabase PostgreSQL


Student
Provider
Frontend (React)
Django API
Supabase PostgreSQL
1 Click Apply on /jobs/{id}
2 POST /api/applications/review/
3 Load profile and active documents
4 200 OK (pre-filled review data)
5 Show mandatory review screen (/apply/{id})
6 Check details, click Confirm
7 POST /api/applications/confirm/
alt
[Already applied (UNIQUE studentid + opportunityid)]
8 Error, duplicate application
9 Show "already applied"
[New application]
10 INSERT application (status Submitted, reference number)
11 201 Created (reference number)
12 Show confirmation with reference number
13 Open listing applicants
14 GET /api/applications/provider/listings/{id}/applicants/
15 200 OK (applicants)
16 Set outcome
17 PATCH /api/applications/provider/applicants/{id}/
18 UPDATE application status and outcome
19 INSERT notification for student
20 200 OK
21 Open My applications
22 GET /api/applications/mine/ and /api/notifications/
23 200 OK (statuses + notifications)
24 Show updated status
Student
Provider
Frontend (React)
Django API
Supabase PostgreSQL

7. Database design

The database is PostgreSQL on Supabase. Ten application tables are created by hand from the data dictionary. Django's own tables (*auth_user , authtoken_token*, sessions) are created by migrations and are separate.

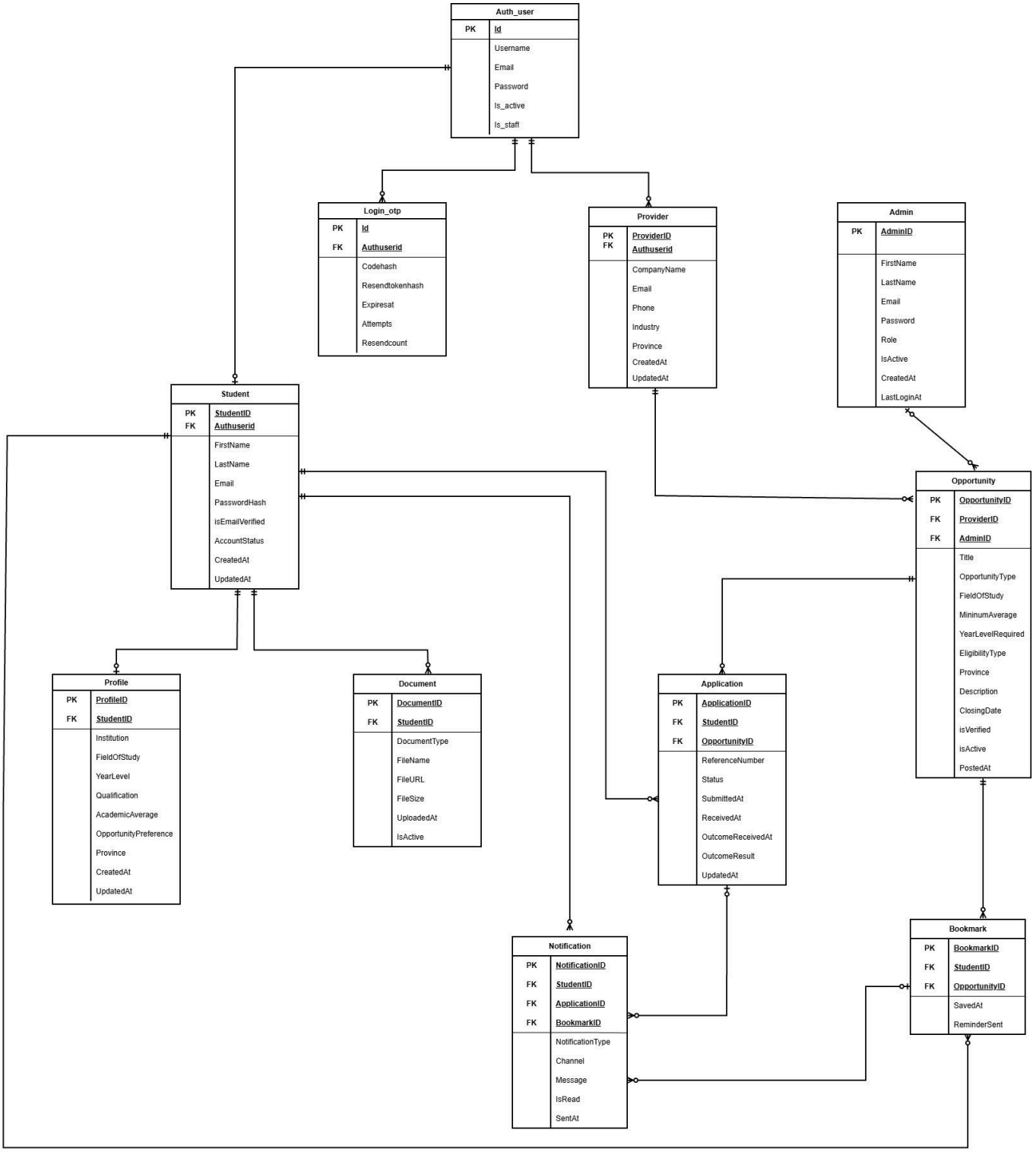


Figure 7: Entity relationship diagram.

6.1 Tables

Table	Purpose
student	One row per registered student
profile	Academic details used for matching
document	Metadata for files in Supabase Storage
provider	Companies posting opportunities
admin	Administrators who verify listings
login_otp	Verification codes issued at signup
opportunity	A listing
application	A student's application
bookmark	A saved opportunity
notification	Receipts, alerts, status updates and reminders

6.2 Relationships

- An *auth_user* links to at most one student and at most one provider, and may have many *login_otp* rows over time.
- A student has one profile, and many documents, applications, bookmarks and notifications.
- A provider has many opportunities. Every opportunity has exactly one provider.
- An administrator has many opportunities through *adminid*, which records who verified the listing.

- An opportunity has many applications and bookmarks. A student can apply to or bookmark a given opportunity once only.
- A notification belongs to a student and optionally to one application or one bookmark.

6.3 Example constraint

The profile table prevents inconsistent data at the database level. A Tertiary Student must have a year level and no qualification. A Graduate must have no year level and a qualification that matches the graduate type: Higher Certificate, Diploma or Bachelor for Undergraduate, and Honours, Masters or Doctorate for Postgraduate. The frontend enforces the same rules, so the database acts as a second safety net.

8. Frontend structure

File or folder	Purpose
src/pages/Landing.jsx	Public home page: hero, about section with tile collage, how it works, POPIA section, calls to action
src/pages/Register.jsx	Student and provider sign-up followed by the six-digit code step
src/pages/Login.jsx	Email and password login.
src/pages/Profile.jsx	Profile form with completion percentage, conditional Tertiary or Graduate fields, document upload cards, and a loading skeleton
src/pages/Profile.jsx	Profile form with completion percentage, conditional Tertiary or Graduate fields, document upload cards, and a loading skeleton
src/pages/Eligible.jsx	Eligible and All toggle with opportunity cards
src/pages/Profile.jsx	Profile form with completion percentage, conditional Tertiary or Graduate fields, document upload cards, and a loading skeleton
src/pages/Eligible.jsx	Eligible and All toggle with opportunity cards

src/pages/JobDetails.jsx, Apply.jsx, Watchlist.jsx	Listing detail, mandatory review and submit, bookmarks

src/pages/Dashboard.jsx, PostListing.jsx, ViewApplicants.jsx, ApplicantDetails.jsx	Provider dashboard, listing form, applicant list and detail
src/pages/AdminVerify.jsx	Administrator verification queue with a detail panel
src/components/	Reusable parts: AppShell, ProviderShell, AuthLayout, RequireAuth (role routing), Navbar, Button, FormField, JobCard and others
src/context/AuthContext.jsx	Token and user state, sign in and sign out
src/lib/api.js	One place for all API calls. Attaches the auth token
src/lib/constants.js	Shared option lists (provinces, qualifications, fields).

7.1 Interface rules

The interface uses an ink, cream and lime palette with Roboto type and lucid icons. Forms validate in the browser first, then show server errors in a single amber banner. Loading states are shown while data is being fetched, so an empty profile is never mistaken for a loaded one.

9.Setup, maintenance and troubleshooting

8.1 Running the system locally

Step	Action

1	Clone the repository and create backend/.env.
2	Create a virtual environment and *run pip install -r requirements.txt*
3	Run python manage.py migrate for Django's own tables
4	Run *database/schema.sql* in the Supabase SQL editor for the application tables
5	Run python *manage.py seed_demo* for demo accounts and listings
6	Run *python manage.py runserver* . The API runs on port 8000
7	In the frontend folder run *npm install* then *npm run dev*. The app runs on port 5173

8.2 Environment variables

Variable	Meaning
DJANGO_SECRET_KEY, DEBUG, ALLOWED_HOSTS	Standard Django settings
DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT	Supabase connection.
DB_SSLMODE, DB_CONN_MAX_AGE	SSL required.
SUPABASE_URL	Project URL only.
SUPABASE_SERVICE_KEY	Storage service key, used by *supabase_storage.py*.
SUPABASE_BUCKET	Storage bucket name.
EMAIL_BACKEND, EMAIL_HOST, EMAIL_PORT	SMTP backend, and the Brevo host on port 587
EMAIL_HOST_USER, EMAIL_HOST_PASSWORD, EMAIL_USE_TLS	Brevo login and key

DEFAULT_FROM_EMAIL	Sender address shown on outgoing email

8.3 Routine maintenance

- Review every pull request before merge. Check ownership filters, parameterised SQL, and that no secrets are included.
- Change the database only by editing *schema.sql* and then applying the same change in Supabase, so the file stays the source of truth.
- Check the Supabase dashboard for usage, paused projects and backups.

10. Testing summary

An end-to-end build verification check exists as a management command: *python manage.py smoke_test.* It walks the full student and provider journey through the real API, prints pass or fail for every step, rolls back all database changes, and deletes files uploaded during the test.

Area	Tests	Results
Build	Frontend production build, Django system check	Pass. Both complete with no errors
Student registration	Register, receive the code by email, verify, land on the dashboard	Pass. Confirmed manually with a real inbox
Email delivery	Brevo SMTP delivers the verification code	Pass. Confirmed manually
Provider registration	Register with company details, verify, reach the provider dashboard	Pass.
Administrator access	Sign in with a staff account, open the queue, verify a listing, confirm the audit field is set	Pass.
Code security	Wrong code rejected, expired code rejected, used code rejected, attempt limit enforced	Pass.
Resend and rate limits	Resend works, cooldown enforced, resend limit enforced, endpoint rate limit enforced	Pass

Login	Correct credentials succeed, wrong credentials rejected, unverified account told to verify	Pass.
Profile	Create and update as Tertiary Student and as Graduate, invalid combinations rejected by the database	Pass.
Documents	Valid upload accepted, wrong type rejected, file over 5 MB rejected, re-upload retires the previous row	Pass.
Eligibility matching	Eligible versus All, unverified and expired listings hidden, eligibility flag correct per criterion	Pass.
Assisted apply	Review screen blocks submission, missing document alert sent once, confirm creates the application with a reference number	Pass.
Duplicate protection	Second application to the same listing rejected, second bookmark has no effect	Pass.
Provider listings	Create, edit and delete own listing. Another provider's listing returns not-found	Pass.
Provider outcomes	Status moves forward only, outcome cannot be changed once set, student is notified	Pass.
Notifications	In-app feed updates, email sent, mark as read works	Pass.
Summary report	PDF generates with correct totals and reference numbers	Pass.
Permissions	A student cannot reach provider or administrator endpoints, and the reverse	Pass.